

Lecture Note on Search Algorithms

COMS 4720/5720: Principles of Artificial Intelligence

Nok Wongpiromsarn

Many decision-making, planning, and puzzle-solving tasks can be formulated as search problems, where an agent must find a sequence of actions that transforms the initial state into a goal state. Search problems have been studied across diverse fields, including artificial intelligence, theoretical computer science, operations research, optimization, and robotics, each of which has developed its own terminology and conventions. Wherever possible, this lecture note draws connections among these perspectives and aligns the terminology to provide a unified view of search concepts. This note complements the material presented in Chapter 3 of our textbook *Russell and Norvig, Artificial Intelligence: A Modern Approach* by expanding on the topic of search. Throughout, it follows the textbook's notation and terminology as closely as possible to maintain consistency. It also assumes that readers are familiar with the foundational concepts and terminology presented in Chapters 1 and 2.

To focus on the core ideas of search, this lecture note considers the simplest kind of environment: fully observable, deterministic, static, discrete, and known. This is the classical setting of problem-solving agents discussed in the textbook. In such environments, the agent knows exactly how each action affects the state, so it can plan an entire sequence of actions in advance to reach the goal. The process of constructing, evaluating, and selecting among these sequences is what we refer to as **search**. A goal-based agent searches for a sequence that brings it to a desired goal state. A utility-based agent extends this notion by associating a utility function with each sequence and searching for one that maximizes overall performance among those that achieve the goal. In contrast, in nondeterministic or partially observable environments, the outcome of an action is uncertain or only partially known, requiring the agent to evaluate multiple possible resulting states and adapt its plan dynamically.

A search algorithm provides a systematic approach for a problem-solving agent to explore possible sequences of actions. It takes as input a search problem and returns either a solution or an indication of failure if no solution exists. In COMS 3110, we encountered this concept in the context of graph search, where states correspond to graph vertices, actions correspond to edges, and the task is to find an (optimal) path from a start vertex to a goal vertex. This lecture note builds on that foundation by studying more general search problems and cost structures that extend beyond explicit graphs. Rather than assuming the entire state space is known in advance, we view search algorithms as incrementally constructing the search space through the application of actions. This perspective unifies a wide range of search and planning algorithms under a common framework.

1 Search Problem

Formulation: A **search problem** is defined by the following components.

- S is the **state space**, i.e., the set of all possible situations the agent may encounter. In general, the state space may be **discrete** (finite or countably infinite) or **continuous** (uncountably infinite). This lecture note specifically considers discrete state space, which may still be infinite. Each element $s \in S$ is referred to as a **state**.
- $s_I \in S$ is the **initial state**, representing the starting point of the search.
- $S_G \subseteq S$ is the set of **goal states**.
- A is the set of all possible **actions**. Each action $a \in A$ transforms one state into another. The set of actions applicable in a state $s \in S$ is denoted by $A(s) \subseteq A$.

- $f : S \times A \rightarrow S$ is the **transition model** (or **state transition function**), which specifies the result of applying an action in a state, i.e., $s' = f(s, a)$ means that applying action a in state s results in the new state s' . We say that s' is a **successor** of s under action a , and conversely, s is a **predecessor** of s' under the same action. Intuitively, the transition model defines how actions move the agent from one state to another within the state space and the predecessor–successor relationship captures the direction of these transitions.
- $c : S \times A \times S \rightarrow \mathbf{C}$ is the **action cost function**, where $c(s, a, s')$ gives the cost of applying action a in state s to reach s' . The set $\mathbf{C}$ represents the set of possible cost values, which may be numerical (e.g., real numbers) or more general structures, depending on the application.

Cost Structures: In this course, we focus on the standard real-valued costs, where $\mathbf{C} = \mathbb{R}$ and costs are combined by ordinary addition:

$$c_{total} = c_1 + c_2.$$

This additive, real-valued formulation covers most classical search problems, where path cost is the accumulated sum of action costs.

However, many real-world applications involve multiple or conflicting objectives that require richer cost representations than a single numeric value can provide. For example, an autonomous vehicle must simultaneously avoid collisions, respect traffic rules, and optimizing performance metrics such as passenger comfort, travel time, distance, and energy consumption. It is often difficult, or even impossible, to define a utility function that combines all these objectives into a single number. For instance, how should we balance passenger comfort against collision avoidance? Different objectives may conflict or operate on different scales, making a single numeric cost insufficient. This motivates the use of multi-objective or structured cost representations, such as vector-valued costs or non-additive aggregation, which allow search algorithms to reason about multiple criteria in a principled way.

To support such applications, we define costs abstractly using a general algebraic framework. Formally, a cost structure is defined as a tuple $(\mathbf{C}, \oplus, \mathbf{1}, \leq)$, where

- (1) $\mathbf{C}$ is the set of all possible costs. It can be any set, not necessarily numeric, and may consist of numbers, vectors, tuples, or other abstract objects, depending on the application.
- (2) $\oplus : \mathbf{C} \times \mathbf{C} \rightarrow \mathbf{C}$ is an associative binary operation used to combine costs.
- (3) $\mathbf{1} \in \mathbf{C}$ is the identity element of $\oplus$, representing zero cost.
- (4) $\leq$ is a reflexive, transitive, and antisymmetric relation on $\mathbf{C}$ used to compare costs.

A quadruple $(\mathbf{C}, \oplus, \mathbf{1}, \leq)$ is known as a **monoid with an order**. This structure provides a consistent way to combine the costs accumulated along an action sequence and to compare total costs across alternative sequences. A cost structure is said to be additive when the operator $\oplus$ corresponds to standard (element-wise) addition.

The real-valued case used in this course is a special case of a monoid with an order, where $\mathbf{C} = \mathbb{R}$, $\oplus$ is ordinary addition, $\mathbf{1} = 0$, and $\leq$ is the usual numeric order. To keep notation simple, this lecture note uses 0 for the identity element and write $+$ instead of $\oplus$. However, all algorithms presented here apply to any cost structure $(\mathbf{C}, \oplus, \mathbf{1}, \leq)$ that forms a **cost monoid**, meaning that it satisfies the following properties

- $(\mathbf{C}, \oplus, \mathbf{1})$ is a monoid, i.e., $\oplus$ is associative and $\mathbf{1}$ is its identity element,
- $\leq$ is a total order on $\mathbf{C}$,
- $\oplus$ is isotone, i.e., for all $x, y, z \in \mathbf{C}$, $x \leq y$ implies $x \oplus z \leq y \oplus z$ and $z \oplus x \leq z \oplus y$, and
- $\mathbf{1}$ is the smallest elements, i.e., $\mathbf{1} \leq x$ for all $x \in \mathbf{C}$.

In this generalized view, any occurrence of addition (+) and zero (0) in the algorithms presented later should be interpreted abstractly as $\oplus$ and $\mathbf{1}$, respectively. This generalization makes it possible to apply classical search algorithms to broader cost structures.

Example 1. For applications with multiple objectives, costs can be represented as vectors, i.e., $\mathbf{C} = \mathbb{R}^n$ for some natural number n , where each component corresponds to each objective or performance metric. The combination operator $\oplus$ can vary across components. Some elements may use standard addition, while others may use non-additive operation such as $x \oplus y = \max(x, y)$ to model worst-case or bottleneck costs. The ordering $\leq$ can be defined lexicographically. This flexible framework allows search algorithms to handle multi-objective optimization and heterogeneous cost aggregation in a unified manner.

Representation of Search Problem for Algorithms: In practice, a search problem is represented by a data structure, denoted here as *problem*, which exposes the components of the problem through the following function interfaces. This allows the algorithm to query the problem definition as needed.

- `INITIAL()` returns the initial state s_I
- `IS-GOAL( $s$ )` returns true if a given state s is a goal state, i.e., $s \in S_G$ and returns false otherwise.
- `ACTIONS( $s$ )` returns the set of applicable actions $A(s)$ in a given state $s \in S$.
- `RESULT( $s, a$ )` returns the successor state $f(s, a)$, resulting from applying action $a \in A(s)$ to state $s \in S$.
- `ACTION-COST( $s, a, s'$ )` returns the cost $c(s, a, s')$ for applying action $a \in A(s)$ in state $s \in S$ to reach successor state $s' \in S$.

Fully Specified vs. Implicit State Spaces: In COMS 3110, we studied search algorithms that operate on a fully specified graph, where all vertices (states) and edges (transitions) are explicitly provided to the algorithm. In that setting, the search operates directly on a complete data structure that represents the entire problem space.

In many applications, however, the state space may be extremely large or even infinite, making it impractical to enumerate all states and transitions. In such cases, search algorithms incrementally build the state space by applying actions from the current states, generating new states as needed. This view treats the state space as implicitly defined by the transition model f , rather than as an explicit graph.

This lecture note adopts the implicit state space perspective, where the structure of the state space is revealed incrementally during search, an approach that underlies most classical AI search algorithms.

Example 2. To make the abstract notion of a search problem more concrete, consider a simple planning problem on a grid world shown in Figure 1. The environment consists of a 4×4 grid, where each cell represents a state identified by its coordinates (x, y) , with the lower-left cell at $(0, 0)$. Two cells, located at $(2, 1)$ and $(2, 2)$ are obstacles, meaning the agent cannot enter or pass through them.

The agent starts in the lower-left corner at state $(0, 0)$ and the goal region consists of two cells $(2, 3)$ and $(3, 3)$. The agent can move up, down, left, or right, as long as the resulting cell lies within the grid and is not an obstacle.

Formalization as a Search Problem: This grid world can be represented as a search problem defined by the following components:

- State space: $S = W \setminus O$, where $W = \{(i, j) \in \mathbb{N} \times \mathbb{N} \mid 0 \leq i, j \leq 3\}$ represents all grid cells and $O = \{(2, 1), (2, 2)\}$ represents the obstacle region.
- Initial state: $s_I = (0, 0)$.
- Goal states: $S_G = \{(2, 3), (3, 3)\}$.

(0,3)	(1,3)	S_G (2,3) (3,3)	
(0,2)	(1,2)	(2,2)	(3,2)
(0,1)	(1,1)	(2,1)	(3,1)
S_I (0,0)	(1,0)	(2,0)	(3,0)

Figure 1: Example 4x4 grid world with obstacles and goal cells.

- Actions: $A = \{(0,1), (0,-1), (-1,0), (1,0)\}$, corresponding to up, down, left, and right moves, respectively. For any state $s \in S$, the set of applicable actions is $A(s) = \{a \in A \mid s + a \in S\}$.
- Transition model: $f(x, a) = x + u$, which maps a state and an action to the resulting state by vector addition.
- Cost function: $c(s, a, s')$ assigns the cost of moving from s to s' via action a . A common choice is a uniform cost, where $c(s, a, s') = 1$ for all valid moves, but more complex variants could penalize proximity to obstacles or prefer certain directions.

Implementation View: From an implementation perspective, this grid world search problem can be represented as a data structure, *problem*, with the following function interfaces:

- *problem*.INITIAL() returns $(0,0)$.
- *problem*.IS-GOAL(s) returns **True** if $s \in \{(2,3), (3,3)\}$ and **False** otherwise.
- *problem*.ACTIONS(s) returns the set of all valid moves from s , i.e., $\{a \in A \mid s + a \in S\}$.
- *problem*.RESULT(s, a) returns the successor state $f(s, a) = s + a$.
- *problem*.ACTION-COST(s, a, s') returns the cost $c(s, a, s')$ of moving from s to s' via action a .

2 Solutions of Search Problems

In a search problem, an agent explores sequences of actions to achieve a goal. To reason formally about these sequences, we introduce several key concepts: paths, solutions, path costs, and optimal solutions. These definitions will allow us to rigorously describe what it means for an agent to solve a search problem and to evaluate the quality of different solutions.

Definition 1 (Path). A **path** is a finite sequence of actions

$$\pi = (a_1, a_2, \dots, a_K)$$

such that $a_i \in A(s_i)$ where $s_1 = s_I$ and $s_{i+1} = f(s_i, a_i)$ for all $i \geq 1$.

Using the transition model f , a path $\pi = (a_1, a_2, \dots, a_K)$ induces a corresponding sequence of states $\mathbf{s}_\pi = (s_1, s_2, \dots, s_{K+1})$, where $s_1 = s_I$ and each successor state is given by $s_{i+1} = f(s_i, a_i)$ for all $i \geq 1$.

Remark 1. In classical graph theory (as covered in COMS 3110), a path is defined as a sequence of states (vertices). In contrast, in AI and planning domains, it is more common to define a path as a sequence of actions. This action-based definition is more general and better suited for modeling decision-making problems, as it allows multiple distinct actions from the same state to result in the same successor state

but with different costs. The corresponding sequence of states can still be recovered by applying these actions successively from the initial state.

For example, in the the grid world problem describe in Example 2, instead of a single move right $(1,0)$ action, we may distinguish between `move_right_fast` (faster but consumes more energy) and `move_right_slow` (slower but consumes less energy). Both actions lead to the same next state but differ in cost, reflecting different trade-offs (e.g., time vs. energy). A purely state-based definition, however, would not capture which specific action was taken between states and therefore could not distinguish between such alternatives.

Definition 2 (Solution). A **solution** of a search problem is a path $\pi = (a_1, a_2, \dots, a_K)$ that transforms the initial state s_I to some goal state in S_G , i.e., $s_{K+1} \in S_G$.

Intuitively, a solution represents a sequence of actions that successfully achieves a desired goal.

Definition 3 (Path Cost). The **cost** of a path $\pi = (a_1, a_2, \dots, a_K)$ is the total accumulated cost of executing its actions

$$c(\pi) = c(s_1, a_1, s_2) + c(s_2, a_2, s_3) + \dots + c(s_K, a_K, s_{K+1}),$$

where $s_1 = s_I$ and $s_{i+1} = f(s_i, a_i)$ for all $i \geq 1$. This gives a quantitative measure of the “effort” associated with executing a particular path.

Definition 4 (Optimal Solution). An **optimal solution** of a search problem is a path π^* whose total cost is minimal among all solutions. Let Π denote the set of all solutions. Then π^* satisfies

$$c(\pi^*) \leq c(\pi), \forall \pi \in \Pi.$$

Optimal solutions are particularly important because they achieve the goal while minimizing cost. In particular, a goal-based agent seeks any solution, whereas a utility-based agent aims specifically to find an optimal solution.

Example 3. Let us revisit the grid world problem describe in Example 2. We now illustrate the notions of path, solution, cost, and optimal solution using a specific example.

Path: Consider a path

$$\pi = ((1,0), (1,0), (1,0), (0,1), (0,1)),$$

i.e., (Right, Right, Right, Up, Up). Using the transition model, this path produces the following sequence of states

$$\mathbf{s}_\pi = ((0,0), (1,0), (2,0), (3,0), (3,1), (3,2)).$$

The final state $(3,2)$ is not a goal state; therefore, π is not a solution.

Solution: A solution is a path that leads from the initial state $s_I = j(0,0)$ to a goal state in $S_G = \{(2,3), (3,3)\}$. Extending the previous path π by one more upward move yields

$$\pi' = ((1,0), (1,0), (1,0), (0,1), (0,1), (0,1)),$$

which generates the sequence of states

$$\mathbf{s}_{\pi'} = ((0,0), (1,0), (2,0), (3,0), (3,1), (3,2), (3,3)).$$

Since the final state $(3,3)$ belongs to the goal set S_G , π' is a solution.

Cost: Suppose each action has a uniform cost of 1. The cost of π' is the number of actions it contains, i.e.,

$$c(\pi') = 6$$

Optimal Solution: An optimal solution is a solution with the minimum total cost among all possible solutions. One such path is

$$\pi^* = ((0,1), (0,1), (0,1), (1,0), (1,0)),$$

which generates

$$\mathbf{s}_{\pi^*} = ((0,0), (0,1), (0,2), (0,3), (1,3), (2,3)).$$

This path reaches a goal state in S_G with 5 actions, so $c(\pi^*) = 5$. Since no shorter path exists, π^* is an optimal solution.

Terminal Cost: In some applications, it may be helpful to introduce a terminal cost $c_F : S \rightarrow \mathbf{C}$. For example, one can define $c_F(s) = 0$ if $s \in S_G$ and $c_F(s) = \infty$ otherwise. This definition allows the concept of optimal solution to be defined without having to define the set Π of all solutions in Definition 4. It also allows the possibility of having goals that are less “crisp” by letting $c_F(s)$ vary for different $s \in S_G$. The total cost of a path $\pi = (a_1, a_2, \dots, a_K)$ is then

$$c(\pi) = \sum_{i=1}^K c(s_i, a_i, s_{i+1}) + c_F(s_{K+1}).$$

3 Performance Measures of Search Algorithms

When evaluating search algorithms, several aspects of performance are typically considered. Different algorithms make trade-offs among these aspects depending on the problem structure and available computational resources.

1. **Completeness:** In general, an algorithm is said to be **complete** if for any input, it is guaranteed to produce a correct result in finite time, returning a solution whenever one exists and correctly reporting failure otherwise. Completeness ensures that the algorithm always terminates with a correct result, although this does not necessarily mean that the solution found is the best or most optimal one.

For search algorithms, completeness means that the algorithm is guaranteed to find a **solution** (See Definition 2) to any given search problem whenever one exists. In a finite state space, a complete search algorithm always terminates in finite time, either returning a solution or correctly reporting failure when no goal is reachable. In a countably infinite state space, this property is often relaxed to eventual completeness, meaning that if a solution exists, the algorithm is guaranteed to return it in finite time, even though it may not terminate when no solution exists.

2. **Cost optimality:** Cost optimality concerns the quality of the solution found, specifically with respect to its total path cost. An algorithm is said to be **optimal** if whenever it returns a solution, that solution is an **optimal solution** (See Definition 4), i.e., one with the minimum path cost among all possible solutions. For optimality to be meaningful, the algorithm must also be complete, since the very notion of finding the best solution assumes that the algorithm will find a solution whenever one exists. Whether a search algorithm is optimal depends on both its search strategy and the assumptions made about the cost structure of the search problem.
3. **Time and space complexity:** Time complexity measures how the number of computational steps required by an algorithm grows with the size of the problem. It reflects the efficiency of the algorithm and determines its practical scalability. Space complexity measures the amount of memory required by an algorithm. It reflects the storage demands of the algorithm and can often be the primary limiting factor in practical applications.

The way we quantify these complexities depends on how the state space is represented. When the state space is fully specified, such as in a graph with explicitly given vertices and edges, complexity is naturally expressed in terms of the size of the graph, i.e., the number of vertices $|V|$ and edges $|E|$, since the algorithm can directly access any state or transition.

For implicit state spaces considered in this lecture note, which are common in AI and robotics, the state space is not enumerated in advance. Instead, new states are generated incrementally by applying actions from currently reached states. In this case, complexity is more naturally expressed in terms of:

- depth (d), i.e., the number of actions in an optimal solution,
- maximum path length (m), i.e., the largest number of actions along any path, and
- branching factor (b), i.e., the maximum number of successor states that can be generated from any single state.

This implicit-state perspective focuses on the number of nodes generated or expanded during search, rather than the size of a pre-constructed graph. It is particularly relevant in AI, where the state space may be very large or infinite, and the algorithm incrementally explores only the relevant portion needed to find a solution.

This lecture note focuses on **systematic search**, a family of algorithms that guarantees **completeness** by exploring the state space in a structured manner. This ensures that every reachable state is eventually considered and that no state is expanded more than once unnecessarily.

In a finite state space, a systematic search must visit every reachable state, allowing it to determine in finite time whether a solution exists. In a countably infinite discrete state space, a systematic search must find a solution in finite time if one exists. If no solution exists, however, the search may continue indefinitely. Formally, systematic search requires that, in the limit as the number of search iterations tends to infinity, every reachable state is eventually reached. This property ensures that the algorithm systematically considers all possibilities without unnecessary repetition, making it complete in finite state spaces and eventually complete in countably infinite state spaces.

4 Search Algorithms

A search algorithm takes a search problem as input and returns either a solution or an indication of failure if no solution exists. Recall that this lecture note adopts the incremental search perspective, where the state space is incrementally revealed as the search progresses. Each algorithm can thus be viewed as constructing a search tree, whose nodes correspond to states reached through different sequences of actions.

Search Tree: A search tree is a conceptual structure that represents how a search algorithm explores the state space.

Each node in the search tree corresponds to a state $s \in S$ and encodes a unique path from the initial state s_I to s . Given any node n , let $\mathbf{s}(n)$ denote the state corresponding to n . Because multiple paths may reach the same state, there can be multiple nodes in the search tree that corresponds to the same underlying state, i.e., there may exist two distinct nodes n, n' with $\mathbf{s}(n) = \mathbf{s}(n')$. Each node thus represents both the state and the sequence of actions taken to reach it. A state is said to be **reached** once at least one node corresponding to it has been generated. The root node of the tree corresponds to the initial state s_I .

Each edge in the search tree corresponds to an action that transitions from the predecessor state to the successor state according to the transition model f . A path from the root of the search tree to any node n corresponds to a sequence of actions to reach state $\mathbf{s}(n)$ from s_I . If $\mathbf{s}(n)$ belongs to the goal set S_G , that path constitutes a solution to the search problem.

Note that a search tree differs from the state space graph. In the state-space graph, each state appears at most once. The search tree, on the other hand, may contain multiple nodes that correspond to the same state, each encoding different paths to that state.

Search algorithms operate by expanding nodes in the search tree according to some search strategy (e.g., depth-first, breadth-first, or best-first). Expanding a node means applying all applicable actions to its corresponding state, generating a new **child node** for each resulting successor state. A node is considered **expanded** once this process has been carried out. Each generated node represents an extended path, with the expanded node as its **parent**. In terms of states, the state corresponding to the parent node is a **predecessor** of the state corresponding to the child node, while the state corresponding to the child node is a **successor** of the state corresponding to the parent node. This correspondence ensures that the tree structure of nodes mirrors the predecessor-successor relationships between states. Each generated node may later be selected for further expansion, depending on the algorithm's strategy.

For systematic search, we can simplify the notion of expansion to the state level. Specifically, we say that a state is **expanded** once one of its corresponding nodes in the search tree has been expanded. Systematic

Algorithm 1 A general template for forward search algorithms

```
1: Function SEARCH(problem, frontier) return a solution node or failure
2:   node  $\leftarrow$  NODE(STATE = problem.INITIAL(), PARENT =  $\emptyset$ , ACTION =  $\emptyset$ , PATH-COST = 0)
3:   if problem.IS-GOAL(node.STATE) then return node
4:   frontier.INSERT(node)
5:   reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node
6:   expanded  $\leftarrow$  an empty lookup table
7:   while not frontier.IS-EMPTY() do
8:     n  $\leftarrow$  frontier.POP()
9:     if problem.IS-GOAL(n.STATE) then return n
10:    s = n.STATE
11:    if s is not in expanded then
12:      expanded[s]  $\leftarrow$  n
13:      for all a in problem.ACTIONS(s) do
14:        s'  $\leftarrow$  problem.RESULT(s, a)
15:        cost  $\leftarrow$  n.PATH-COST + problem.ACTION-COST(s, a, s')
16:        n'  $\leftarrow$  NODE(STATE = s, PARENT = n, ACTION = a, PATH-COST = cost)
17:        if s' is not in reached then
18:          reached[s']  $\leftarrow$  n'
19:          frontier.INSERT(n')
20:        else if s' is not in expanded then
21:          frontier.RESOLVE-DUPLICATE(n', frontier, reached)
```

search algorithms ensure that each state is expanded at most once by preventing the generation of new nodes for states that have already been expanded. This convention provides an unambiguous way to measure progress through the state space.

Despite their differences, all classical search algorithms share the same general framework:

1. Select which node to expand according to the search strategy.
2. Generate new nodes incrementally by expanding the chosen node (applying actions to its corresponding state).
3. Keep track of the paths (and their associated costs).

This unified view allows diverse search algorithms to be expressed as variations of a common search template shown in Algorithm 1.

Node: A node in the search tree can be represented by a data structure with four components:

- STATE: the state corresponding to the node;
- PARENT: the node in the tree that generated this node;
- ACTION: the action that was applied to the state corresponding to the parent node to generate this node;
- PATH-COST: the total cost of the path from the root of the search tree to this node.

Frontier of the search tree: The variable *frontier* represents the set of generated leaf nodes that are unexpanded and eligible for expansion. In other words, *frontier* contains nodes that have been generated but not yet expanded and define the boundary between the portion of the state space that has already been expanded and the portion that has not yet been reached. Note that some leaf nodes of the search tree may not be part of the frontier if they are known to be ineligible for expansion.

Efficient management of the frontier requires a data structure that supports the following operations:

- `IS-EMPTY()` returns true if and only if the frontier contains no nodes
- `POP()` removes and returns the next node to be considered for expansion, according to the search strategy (e.g., FIFO for BFS).
- `INSERT( $n$ )` inserts a node n into the frontier.
- `RESOLVE-DUPLICATE( $n, frontier, reached$ )` handles the case where node n corresponds to a state that has already been reached but not yet expanded. The specific action taken depends on the algorithm's duplicate-handling policy.

The key difference between search algorithms lies in how they order the frontier and how they handle duplicate states. Specifically, each algorithm defines a different function for prioritizing nodes in *frontier* and a different policy in `RESOLVE-DUPLICATE`.

State Classification During Search: During the execution of a search algorithm, each state can be classified based on whether it has been reached (i.e., at least one node has been generated for it) or expanded (i.e., one of its corresponding nodes has been expanded). This classification provides a clear measure of the algorithm's progress through the state space.

Recall that a node n represents a path from the initial state *problem.INITIAL* to the state $s(n)$. Multiple nodes may correspond to the same underlying state if there are multiple distinct paths that reach it.

At any point during the search, each state $s \in S$ falls into one of the following categories:

1. *unvisited*: The state s has not been reached, i.e., no node n in the search tree satisfies $s(n) = s$. Initially, all states except the initial state *problem.INITIAL* are *unvisited*.
2. *alive*: The state s has been reached but not yet expanded. These are the states that are active in the search and may still generate new nodes for their successor states when expanded. Initially, the only *alive* state is the initial state *problem.INITIAL*.
3. *dead*: The state s has been expanded. s is considered “dead” because there is nothing more that it can contribute to the search.

Equivalently, a state with no corresponding node generated is *unvisited*. A state with at least one corresponding node in *frontier* and no corresponding node removed from *frontier* is *alive*. A state with at least one corresponding node removed from *frontier* is *dead*.

Remark 2. An alternative (tree-based) convention defines a state as *dead* only when **all** generated nodes for that state have been expanded. Under this view, an *alive* state has at least one generated node in the *frontier*, while a *dead* state has no generated nodes in the frontier. These two conventions differ only in how they treat duplicate nodes for the same state. Our pop-based (graph-search) convention is commonly used in algorithms that maintain a closed/expanded set and intentionally avoid re-expanding a state once it has been expanded (e.g., BFS, DFS, Dijkstra, A*). The tree-based convention is more natural when reasoning purely in terms of search trees, where each node is treated independently.

Open and Closed Lists: In much of the search literature, algorithms are often described as maintaining an **open list** and a **closed list**. Conceptually, these lists partition the search process into what remains to be expanded and what has already been expanded.

The open list contains nodes that have been generated but not yet expanded. Since multiple nodes may correspond to the same underlying state, the open list can contain multiple nodes that correspond to the same state. As a result, it may temporarily include nodes whose states have already been expanded through other paths. Such outdated nodes are typically detected and discarded when they are removed from the open list.

The closed list stores nodes that have already been expanded. Each node in the closed list corresponds to a state that has been expanded at least once, but multiple nodes in the closed list may correspond to the same underlying state if that state was reached by different paths. However, in many algorithms, the closed list is used to enforce a state-level invariant, i.e., no state is expanded more than once. In that case, the closed list can be viewed as implicitly defining the set of states that have already been expanded.

In our framework, the open list corresponds to the variable *frontier*, while the set of *dead* states corresponds to the state-level abstraction of the closed list. This interpretation clarifies that even if the open list may temporarily contain nodes whose corresponding states are already dead (due to duplicates), only the first expansion of each state contributes to the search's progress through the state space.

In practice, the open list is often implemented as a priority queue (or another ordered container), while the closed list is often implemented as a hash table or set that records nodes (or states) that have already been expanded.

Informed and Uninformed Search: Search algorithms can be broadly categorized into **uninformed** and **informed** methods, depending on whether they use additional knowledge about the problem domain. Uninformed search algorithms do not rely on any domain-specific information beyond what is explicitly provided in the problem definition. In other words, they operate solely on the information available in the *problem* data structure, without any estimate of how close a state is to the goal. In contrast, informed search algorithms exploit domain knowledge, typically in the form of a heuristic function, to guide the search toward promising regions of the state space more efficiently. Uninformed methods provide the conceptual foundation for understanding search behavior, while informed methods build on these principles to improve efficiency and scalability.

Example 4. Consider again the 4×4 grid world search problem in Example 2. In each stage of the search, we will show the search tree (right) and the corresponding state classification (left). White cells are unvisited (no node generated), green cells are alive (node is generated, pending expansion), and red cells are dead (expanded).

Stage 1 (Initialization): The root node is generated and is inserted into the frontier. Its corresponding state (the initial state) becomes *alive*.

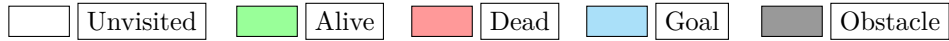
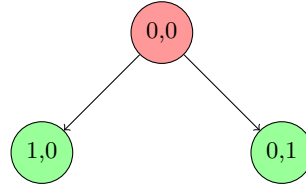
(0,3)	(1,3)	(2,3)	(3,3)
(0,2)	(1,2)	(2,2)	(3,2)
(0,1)	(1,1)	(2,1)	(3,1)
(0,0) alive	(1,0)	(2,0)	(3,0)

0,0

	Unvisited		Alive		Dead		Goal		Obstacle
--	-----------	--	-------	--	------	--	------	--	----------

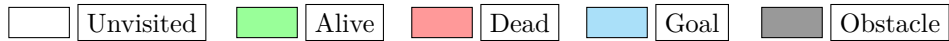
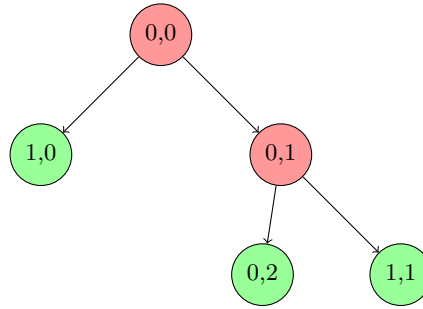
Stage 2: The initial state is expanded and becomes *dead*. Its successor states $(1, 0)$ and $(0, 1)$ are reached for the first time. Their corresponding nodes are generated and inserted into the frontier, making these states *alive*.

$(0,3)$	$(1,3)$	$(2,3)$	$(3,3)$
$(0,2)$	$(1,2)$	$(2,2)$	$(3,2)$
$(0,1)$ alive	$(1,1)$	$(2,1)$	$(3,1)$
$(0,0)$ dead	$(1,0)$ alive	$(2,0)$	$(3,0)$



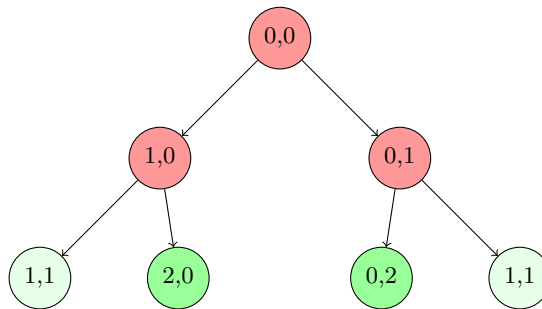
Stage 3: The search algorithm selects one node from the frontier for expansion. Suppose it chooses the node corresponding to state $(0, 1)$. This node is then expanded, making the state $(0, 1)$ *dead*. Its successor states $(1, 1)$ and $(0, 2)$ are *reached*. Their corresponding nodes are generated and inserted into the frontier, making these states *alive*. The frontier now contains nodes corresponding to states $(1, 0)$, $(0, 2)$, and $(1, 1)$.

$(0,3)$	$(1,3)$	$(2,3)$	$(3,3)$
$(0,2)$ alive	$(1,2)$	$(2,2)$	$(3,2)$
$(0,1)$ dead	$(1,1)$ alive	$(2,1)$	$(3,1)$
$(0,0)$ dead	$(1,0)$ alive	$(2,0)$	$(3,0)$



Stage 4: Next, suppose the algorithm expands the node corresponding to state $(1, 0)$. This expansion generates a new node for state $(1, 1)$, resulting in 2 distinct nodes corresponding to the same state, one reached via $(0, 1)$ and the other via $(1, 0)$. This is known as a **redundant path**.

$(0,3)$	$(1,3)$	$(2,3)$	$(3,3)$
$(0,2)$ alive	$(1,2)$	$(2,2)$	$(3,2)$
$(0,1)$ dead	$(1,1)$ alive*	$(2,1)$	$(3,1)$
$(0,0)$ dead	$(1,0)$ dead	$(2,0)$ alive	$(3,0)$



5 Best-First Search

Best-first search is a general search strategy that expands nodes in order of increasing value of an evaluation function $f(n)$. The function $f(n)$ assigns a numerical score to each node n , estimating how promising it is to expand n next. The choice of $f(n)$ determines the behavior of the algorithm and unifies several well-known search strategies under the same framework.

In this case, the variable *frontier* is a priority queue ordered by increasing $f(n)$. At each iteration, the algorithm selects the node n with the smallest $f(n)$ from the frontier and expands it. The operation *frontier*.RESOLVE-DUPLICATE(n , *frontier*, *reached*) updates *reached*[n .STATE] to n and add n to *frontier* if the new path has lower cost (i.e., n .PATH-COST < *reached*[n .STATE].PATH-COST). Otherwise, it does nothing.

Greedy best-first search is a special case of best-first search in which the evaluation function depends only on the estimated cost to reach the goal:

$$f(n) = h(n),$$

where $h(n)$ is a heuristic function estimating the remaining cost from the node's state to a goal state. Greedy best-first search always selects the node that appears to be closest to the goal according to the heuristic, without considering the cost already incurred to reach it. As a result, it tends to find a goal quickly in many cases, but it is not guaranteed to find an optimal solution, and it may even fail to find a solution in infinite or poorly guided search spaces. Some authors use the term “best-first search” specifically to refer to this heuristic-driven variant ($f(n) = h(n)$), rather than the more general formulation that allows any evaluation function.

6 Uninformed Search Strategies

Uninformed search algorithms, also known as “blind search” methods, rely only on the information contained in the *problem* data structure, namely the initial state (*problem*.INITIAL()), goal test (*problem*.IS-GOAL(s)), actions (*problem*.ACTIONS(s)), transition model (*problem*.RESULT(s, a)), and action cost (*problem*.ACTION-COST(s, a, s')), to determine which states to expand next. While often inefficient for large or complex problems, uninformed methods are conceptually fundamental and provide a baseline for evaluating more sophisticated, informed search algorithms. Common examples include BFS, DFS, Depth-Limited Search, Iterative Deepening Search (IDS), and Uniform-Cost Search (UCS). These strategies illustrate the trade-offs between systematic exploration, memory usage, and solution optimality.

In this section, we describe specific uninformed search algorithms using the general search template presented in Algorithm 1, specifying how the frontier is ordered and how duplicate states are handled for each algorithm. We also show how each algorithm can be seen as a special case of best-first search (Section 5), with the evaluation function $f(n)$ defined according to the algorithm's strategy.

6.1 Breadth-First Search (BFS)

BFS expands nodes in a layered manner, exploring all nodes at a given depth before moving to the next. In terms of a search tree, this corresponds to expanding nodes in the order in which they are generated.

Implementation of the General Search Template: For BFS, the variable *frontier* can be implemented as a FIFO queue, so the node that has been in the frontier the longest is selected when *frontier*.POP() is called. We can formalize the order of node expansion in BFS using a priority function defined on nodes in the frontier. Each node is assigned a value equal to the order in which it was inserted into *frontier*, with lower values corresponding to higher priority.

The operation *frontier*.RESOLVE-DUPLICATE(n , *frontier*, *reached*) does nothing. Thus, for BFS, each state has at most one corresponding node.

From the perspective of classical BFS terminology, states that are marked as “explored” or “visited” are either *alive* if it has a corresponding node in the frontier or *dead* otherwise.

A Special Case of Best-First Search: BFS can also be viewed as a specific instantiation of best-first search, where the evaluation function $f(n)$ is simply the depth of node n , i.e., the number of actions required to reach n from the root of the search tree. This ensures that nodes closer to the root (shallower nodes) are always expanded first.

Completeness and Optimality: BFS is systematic, meaning it will find a solution in finite time if one exists, both for finite and countably infinite state spaces. It is cost-optimal when all actions have the same cost, i.e., there is a constant K such that $c(s, a, s') = K$ for all $s \in S$, $a \in A(s)$, and $s' \in f(s, a)$.

Time and Space Complexity: The time and space complexity of BFS is $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. Each layer l of the BFS tree contains b^l nodes, so the total number of nodes up to depth d is $O(b^d)$. Memory usage is often the limiting factor in BFS, as the frontier can grow exponentially with the depth of the solution.

6.2 Depth-First Search (DFS)

DFS explores as far as possible along a single branch before backtracking. In terms of a search tree, this corresponds to expanding nodes in the reverse order in which they are generated, so that the most recently generated node is expanded first.

Implementation of the General Search Template: For DFS, the variable *frontier* is implemented as a stack (i.e., LIFO queue), so the node that has been in the frontier the shortest is selected when *frontier*.POP() is called. As in BFS, we can assign each node a value equal to the order in which it is inserted into *frontier*. For DFS, however, nodes with higher values have higher priority, reflecting the stack behavior.

The behavior of *frontier*.RESOLVE-DUPLICATE($n, frontier, reached$) depends on how the algorithm tracks visited states. Under the “mark-on-pop” convention, states are marked as “explored” or “visited” only when one of their corresponding nodes are removed from the frontier (i.e., when the state becomes *dead*). In this case, the operation *frontier*.RESOLVE-DUPLICATE($n, frontier, reached$) simply updates *reached*[n .STATE] to n and adds n to *frontier* (i.e., using *frontier*.INSERT(n)) without removing or updating any existing nodes. As a result, multiple nodes may correspond to the same underlying state, each reached via a different parent. The actual parent of a state s is determined by the first node corresponding to s that is removed (popped) from *frontier* (i.e., when s becomes dead).

Under the “mark-on-push” convention, states are marked as “explored” or “visited” as soon as their corresponding nodes are generated. In this case, the operation *frontier*.RESOLVE-DUPLICATE($n, frontier, reached$) does nothing, just as in BFS because duplicates are never inserted into the frontier. This approach is sufficient for search problems which only concern with finding a path from the initial state to a goal state.

The mark-on-pop convention, however, is essential in applications that require exploring each path as deeply as possible before backtracking, such as in topological sorting, computing strongly connected components in directed graphs, and other algorithms that rely on depth-first traversal order. By delaying the marking of visited states until expansion, the algorithm allows deep, systematic exploration of each branch, ensuring that all reachable nodes are processed in proper depth-first order.

A Special Case of Best-First Search: DFS can also be viewed as a specific instantiation of best-first search, where the evaluation function $f(n)$ is simply the negative of the depth of node n , meaning that nodes with larger depth are given higher priority. This ensures that search always expands the most

recently generated node first, effectively exploring one path as deeply as possible before backtracking to explore alternative paths.

Completeness and Optimality: DFS is systematic only for finite state space. However, it is not systematic for infinite state space because it may follow a single path indefinitely (e.g., down an infinite corridor), leaving other reachable states unexplored.

DFS is not cost-optimal since it may find a solution far from minimal cost even if a lower-cost solution exists elsewhere in the state space.

Time and Space Complexity: The time complexity of DFS is $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree, because in the worst case it may explore all nodes along the deepest path.

The space complexity, on the other hand, is much smaller than that of BFS and is one of the main advantages of DFS. If the state space contains no cycles, there is no need to maintain a *reached* or *dead* table. Moreover, DFS can generate successors one at a time, storing only the current path from the root to the node being expanded. This variant, known as **backtracking search** and typically implemented recursively, requires only $O(m)$ space, since it stores at most one node per depth level along the current search path.

If cycles exist, a *reached/dead* table is required to avoid revisiting dead states, increasing space complexity to $O(bm)$ in the worst case.

Depth-Limited Search: To prevent DFS from following an infinite path, we can use depth-limited search, a variant of DFS where nodes beyond a specified depth limit l are treated as having no successors. This reduces the time complexity to $O(b^l)$ and the space complexity to $O(bl)$. However, imposing a depth limit may compromise completeness if the limit is too small.

A natural choice for the depth limit is the **diameter** of the state space, defined as the length of the longest shortest path between any two reachable states. In a finite state space, standard DFS with full backtracking will eventually explore all paths up to this depth. Therefore, setting the depth limit to the diameter ensures that DFS will reach any goal state and remains complete, while still benefiting from the space savings of depth-limiting. For infinite state spaces, completeness is not guaranteed since there may exist paths of unbounded length.

Iterative Deepening Search: When the diameter or solution depth is unknown, a more practical approach is **iterative deepening search** (IDS). The core idea is to repeatedly runs depth-limited search with increasing depth limits ($l = 0, 1, 2, \dots$) until a solution is found. Although this involves re-expanding nodes at shallower depths multiple times, IDS maintains the memory efficiency of DFS.

Specifically, in state space with no cycles, each state is generated at most once per iteration. The total number of node expansions over all iterations is $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. Thus, IDS achieves the same asymptotic time complexity as BFS, while requiring only $O(bd)$ space (or $O(d)$ space when using backtracking search). For finite state space, it is complete and cost-optimal when all step costs are equal. If no solution exists, IDS continues until the search depth exceeds the diameter of the state space. In that case, the time complexity becomes $O(b^m)$, where m is the maximum depth explored (or the diameter in a finite space), and the algorithm remains complete for finite but not infinite state spaces.

For state spaces that contain cycles, states can be revisited through different paths. To prevent infinite loops, each iteration of IDS must maintain a *reached/dead* table to record states already expanded within the current depth limit. With duplicate detection, IDS remains complete for finite state spaces and still uses $O(bd)$ space. The time complexity also remains $O(b^d)$, though in dense cyclic graphs the practical runtime may exceed that of BFS.

6.3 Dijkstra's algorithm

Dijkstra's algorithm is known as **uniform-cost search** (UCS) in the AI community because it spreads out the waves of uniform path-cost. It is a form of best-first search with $f(n) = n.\text{PATH-COST}$. In other words, nodes are expanded in order of increasing path cost.

Implementation of the General Search Template: For Dijkstra's algorithm, the variable *frontier* is implemented as a priority queue sorted by the PATH-COST value of the nodes. At each iteration, the algorithm removes from *frontier* the node with the smallest PATH-COST (breaking ties arbitrarily), expands it by generating all its child nodes, and inserts each child node into *frontier* if its corresponding state has not yet been expanded or if a cheaper path to that state has been found.

Formally, we can view the algorithm as incrementally computing a function

$$C : S \rightarrow \mathbf{C} \cup \{\infty\},$$

known as the **cost-to-come**, representing the best (lowest) known cost to reach each state. Let $\mathbf{N}$ denote the set of nodes currently in the search tree. For each state $s \in S$, $C(s)$ is defined as the minimum path cost among all nodes in $\mathbf{N}$ that correspond to s :

$$C(s) = \min_{n \in \mathbf{N} \text{ such that } s(n)=s} n.\text{PATH-COST},$$

and $C(s) = \infty$ if no node corresponding to s has been generated (i.e., s is *unvisited*).

Initially, $C(s_I) = 0$ and $C(s) = \infty$ for all $s \neq s_I$. When a node n is expanded, each resulting child node n' is constructed by applying an action a to $s(n)$. If n' provides a cheaper path to its corresponding state than previously known, i.e., $n'.\text{PATH-COST} < C(s(n'))$, then cost-to-come to $s(n')$ is updated to

$$C(s(n')) = n'.\text{PATH-COST}.$$

Thus, the function C is computed incrementally as nodes are generated and expanded.

By making sure that *reached*[s] is the node with minimum PATH-COST among all the nodes corresponding to s , we can obtain cost-to-come from variable *reached*:

$$C(s) = \begin{cases} \text{reached}[s].\text{PATH-COST} & \text{if } s \text{ is in } \text{reached} \\ \infty & \text{otherwise} \end{cases}$$

A simple implementation of the operation *frontier.RESOLVE-DUPLICATE*($n, \text{frontier}, \text{reached}$) updates *reached*[$s(n)$] to n and adds n to *frontier* if $n.\text{PATH-COST} < C(s(n))$. Otherwise, it does nothing. This allows multiple nodes corresponding to the same underlying state to coexist in *frontier* and the search tree. When the first node corresponding to a state s is popped from *frontier* (i.e., when s becomes *s*(*dead*)), *reached*[s].PATH-COST is guaranteed to be the optimal cost to reach s . All other nodes corresponding to s will necessarily have equal or higher path costs and can be ignored.

To save memory, an alternative implementation of *frontier.RESOLVE-DUPLICATE*($n, \text{frontier}, \text{reached}$) simply updates the existing node n' in *frontier* that corresponds to the same state as n (i.e., $s(n') = s(n)$). Instead of adding n to *frontier*, it updates the fields $n'.\text{PARENT}$ to $n.\text{PARENT}$ and $n'.\text{PATH-COST}$ to $n.\text{PATH-COST}$ and reorders *frontier* accordingly if $n.\text{PATH-COST} < n'.\text{PATH-COST}$. This ensures that each state $s \in S$ has at most one corresponding node in the search tree and is what most textbook Dijkstra implementations use. While a node n is still in *frontier*, $n.\text{PATH-COST}$ may be lower and $n.\text{PARENT}$ may be updated. But once n is popped from *frontier*, the state $s(n)$ is *dead* and $n.\text{PATH-COST}$ and $n.\text{PARENT}$ are final and optimal. This optimal cost is the minimum cost among all the possible paths from s_I to *nodestaten*.

A Special Case of Best-First Search: DFS can also be viewed as a specific instantiation of best-first search, where the evaluation function $f(n) = n.\text{PATH-COST}$, meaning that nodes with smaller PATH-COST are given higher priority. This ensures that search always expands the node with minimum PATH-COST first.

Completeness and Optimality: Dijkstra’s algorithm is both complete and cost-optimal, provided that the action cost function is non-negative, i.e., $c(s, a, s') \geq 0$ for all $s \in S$, $a \in A(s)$, and $s' = f(s, a)$, and that the branching factor is finite. Moreover, if all action costs are strictly positive, the algorithm will not follow an infinite path, unlike DFS.

The completeness and optimality of Dijkstra’s algorithm also extend to more general cost structures $(\mathbf{C}, \oplus, \mathbf{1}, \leq)$ that forms a **cost monoid**. The usual real-valued case with non-negative costs is simply a special case of this structure with $\mathbf{C} = \mathbb{R}_{\geq 0}$, $\oplus = +$, $\mathbf{1} = 0$, and $\leq$ is the standard order. Since 0 is the smallest element in $\mathbb{R}_{\geq 0}$ and addition is isotone, the cost structure satisfies the monoid properties. Interestingly, with $\oplus$ being the max operation instead of addition, $(\mathbb{R}_{\geq 0}, \oplus, 0, \leq)$ still forms a valid cost monoid since max is associative, isotone, and has identity 0. This variant can represent bottleneck or “worst-case” path costs, where the cost of a path is determined by the most expensive single step.

A more general example is a multi-objective cost monoid where $\mathbf{C} = \mathbb{R}_{\geq 0}^n$ for some positive integer n . Each component corresponds to an individual objective, $\oplus$ is defined as element-wise addition, $\mathbf{1}$ is the zero vector, and $\leq$ corresponds to lexicographical order. This structure also forms a valid cost monoid. However, if $\oplus$ involves using max operation for some of the components, the isotonicity may no longer hold, breaking the monoid structure and invalidating Dijkstra’s optimality guarantees.

Time and Space Complexity: The time and space complexity of Dijkstra’s algorithm is $O(b^{1+\lceil C^*/\epsilon \rceil})$, where C^* is the cost of the optimal solution and $\epsilon > 0$ is a lower bound on the cost of each action. The exponent $1 + \lceil C^*/\epsilon \rceil$ reflects the fact that Dijkstra’s algorithm expands nodes in order of increasing path cost. Each layer of the search corresponds to a range of cumulative path costs of width at least ϵ . Thus, before reaching the optimal solution of cost C^* , the algorithm may generate all nodes whose path costs are less than or equal to C^* , spanning roughly C^*/ϵ cost layers. The extra factor of b accounts for the expansion of all successors of the last set of nodes just before the optimal solution is found, resulting in the overall bound $O(b^{1+\lceil C^*/\epsilon \rceil})$.

When all action costs are equal, $\lceil C^*/\epsilon \rceil = d$ so the time and space complexity reduces to $O(b^d)$, the same as BFS.

7 Informed Search

Informed search algorithms use additional knowledge about the problem, typically in the form of a heuristic function $h(n)$, to guide exploration toward the goal more efficiently. The heuristic function $h(n)$ estimates the cost of the cheapest path from $s(n)$ to a goal state in S_G .

A simple example is **greedy best-first search**, which expands the node that appears closest to the goal according to $h(n)$. Because it relies only on the heuristic estimate and ignores the cost to reach the node, greedy best-first search often finds solutions quickly but its solutions may not be optimal.

A more balanced approach is A* algorithm, which combines cost-to-come, denoted $g(n)$, analogous to the function $C(s)$ used in Dijkstra’s algorithm, with the heuristic estimate to get the estimated total cost $f(n) = g(n) + h(n)$. By prioritizing nodes with lower $f(n)$, A* focuses exploration on promising paths while still ensuring completeness and optimality when h is **admissible** (i.e., never overestimates the true remaining cost) or, more strongly, **consistent** (i.e., satisfies the triangle inequality).

Please read Section 3.5 and 3.6 of the textbook for more details.

8 Bidirectional Search

Bidirectional search reduces search effort by simultaneously exploring from both the initial state s_I and a goal state $s_G \in S_G$. It maintains 2 frontiers, one for the forward search and one for the backward search. The search terminates successfully when the two frontiers meet, forming a continuous path from s_I to s_G .

Because each search only needs to explore roughly half the distance to the goal, the total number of expanded nodes can be reduced from $O(b^d)$ to $O(b^{d/2})$ in the best case, where d is the solution depth. Bidirectional variants of Dijkstra's and A* can guarantee optimality when both directions use consistent cost estimates and the meeting condition is defined appropriately.

References

This note combines material from the following sources:

1. S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach* (fourth edition), Pearson, 2020.
2. S. M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms* (fourth edition), MIT Press, 2022.
4. K. Slutsky, D. Yershov, T. Wongpiromsarn, and E. Frazzoli, "Hierarchical multiobjective shortest path problems," in *Algorithmic Foundations of Robotics XIV*, S. M. LaValle, M. Lin, T. Ojala, D. Shell, and J. Yu, Eds. Cham, Switzerland: Springer International Publishing, 2021, pp. 261–276.